

Hợp nhất dữ liệu giao thông đa nguồn tại các node biên và đánh giá hiệu năng kiến trúc Node-Agent-Edge cho bài toán **giám sát giao thông đô thị**.

Camera (**TomTom, OpenStreetMaps, ... = dữ liệu thô**)--> **Tiền xử lý** --> **AI Detection & phân loại** --> Tính mật độ D, vận tốc V --> dựa trên **Hợp nhất dữ liệu loại trùng/có trọng số** --> **Hiển thị bản đồ giao thông Web / app** → **Đánh giá Hiệu năng hệ thống**

1) Tư tưởng hiện thực tổng thể

triển khai hệ thống theo **3 lớp rõ ràng**: 3 module

Lớp 1. **Edge** thiết bị sensor , cam1, camN, ...

Mỗi **node biên** là một vùng quản lý giao thông, ví dụ:

- một ngã tư,
- một đoạn đường,
- một khu vực giám sát.

Mỗi node chứa:

- 1 hoặc nhiều camera,
- **AI Agent** cục bộ,
- module xử lý ảnh/video,
- module hợp nhất dữ liệu cục bộ,
- module gửi kết quả lên server trung tâm.

Lớp 2. Agent : Q-Learning or GNN-GRU

Đây là lớp “thông minh” tại mỗi node:

- nhận dữ liệu từ nhiều camera,
- phát hiện xe,
- theo dõi xe,
- ước lượng tốc độ, mật độ,
- xử lý vùng chồng lấn,
- hợp nhất dữ liệu thành một trạng thái giao thông thống nhất của node.

Lớp 3. Central/Cloud Layer

Server trung tâm dùng để:

- thu nhận kết quả từ nhiều node,
- hiển thị bản đồ giao thông,
- lưu trữ dữ liệu,
- đánh giá hiệu năng toàn hệ thống,
- so sánh các cấu hình Node-Agent-Edge.

2) Pipeline chi tiết từ đầu đến cuối

Giai đoạn A. Thu nhận dữ liệu đa camera (TomTom/ gg maps)

A1. Input

Mỗi camera C_i gửi:

- video stream RTSP/IP camera,
- timestamp,
- camera ID,
- thông tin vị trí lắp đặt,
- góc nhìn/FOV,
- vùng quan tâm ROI.

A2. Đồng bộ thời gian

Cần đồng bộ:

- timestamp giữa các camera,
- FPS xử lý,
- mốc thời gian gửi dữ liệu lên node.

A3. Dữ liệu cấu hình camera

Mỗi camera cần có file cấu hình:

- camera_id
- node_id
- location
- ROI
- homography matrix hoặc tham số ánh xạ ảnh --> mặt đường
- vùng chồng lấn với camera khác
- hệ số tin cậy ban đầu.

Giai đoạn B. Tiền xử lý tại edge

Đây là khối “Tiền xử lý” trong hình.

B1. Chuẩn hóa khung hình

- resize ảnh/video về kích thước thống nhất,
- giảm nhiễu,
- cân bằng sáng nếu cần,
- loại bỏ frame mờ nặng.

B2. Cắt vùng quan tâm ROI

Chỉ giữ vùng đường cần phân tích để:

- giảm tải tính toán,
- tránh nhiễu từ nhà cửa, bầu trời, người đi bộ ngoài vùng nghiên cứu.

B3. Kiểm tra chất lượng ảnh

Tính các chỉ số:

- độ sáng,
- độ nét,
- tỷ lệ che khuất,
- độ rung,
- thời tiết xấu.

Các chỉ số này sẽ dùng sau cho **trọng số hợp nhất**.

Ví dụ:

$$q_i = f(\text{brightness, blur, occlusion})$$

Giai đoạn C. AI Detection và phân loại phương tiện **gọi thư viện Yolo và Pytorch và OpenCV**

Đây là khối “AI Detection phát hiện xe và phân loại”.

C1. Phát hiện đối tượng

Dùng mô hình như:

- YOLOv8 / YOLOv10 / RT-DETR / EfficientDet

Đầu ra mỗi frame:

- bounding box,
- class: xe máy, ô tô, xe buýt, xe tải,
- confidence score.

C2. Tracking

Để tránh đếm lặp giữa các frame, phải có tracking:

- ByteTrack
- DeepSORT
- OC-SORT

Đầu ra:

- mỗi xe có track_id,
- lịch sử chuyển động qua các frame.

C3. Phân loại phương tiện chuẩn hóa

Chuẩn hóa về nhóm:

- motorcycle
- car
- bus
- truck

Có thể gán hệ số quy đổi:

- xe máy = 1
- ô tô = 4
- bus = 16
- truck = 16

hoặc bộ hệ số khác nếu thầy muốn phù hợp thực tế Việt Nam.

3) Ước lượng thông số giao thông tại từng camera của Tomton

Đây là khối “Tính toán mật độ D và vận tốc V”.

D1. Tính mật độ giao thông D_i cho từng camera khi ra hội đồng viết Công thức

Giả sử với camera C_i :

$$N_{eq}^{(i)} = n_{mc}^{(i)} + 4n_{car}^{(i)} + 16n_{bus}^{(i)} + 16n_{truck}^{(i)}$$

$$D_i = \frac{N_{eq}^{(i)}}{A_i}$$

Trong đó:

- A_i : diện tích vùng quan sát đã hiệu chỉnh về mặt đường,
- $N_{eq}^{(i)}$: số lượng phương tiện tương đương.

D2. Tính vận tốc V_i cho từng camera

Có 2 cách hiện thực tốt:

Cách 1: Dựa trên tracking + hiệu chỉnh phối cảnh

- lấy quỹ đạo của xe qua nhiều frame,
- ánh xạ tọa độ ảnh sang mặt phẳng đường,
- tính quãng đường thực,
- chia cho thời gian.

$$v_k = \frac{\Delta s_k}{\Delta t_k}$$

Sau đó tính vận tốc trung bình camera:

$$V_i = \frac{\sum_k c_k v_k}{\sum_k c_k}$$

Trong đó c_k có thể là trọng số theo độ tin cậy track.

Cách 2: Dùng line-crossing + calibration

- thiết lập 2 vạch ảo trên đường,
- đo thời gian xe đi qua hai vạch,
- suy ra vận tốc.

Cách 2 dễ hiện thực hơn nếu thiếu calibration sâu. **paper**

4) Xử lý vùng chồng lấn giữa các camera

Đây là phần rất quan trọng trong sơ đồ

E1. Xác định vùng chồng lấn

Với mỗi cặp camera:

- định nghĩa polygon vùng chồng lấn,
- hoặc ánh xạ về tọa độ mặt đường chung rồi tính giao nhau.

E2. Phát hiện đối tượng trùng

Một xe xuất hiện ở 2 camera có thể bị đếm 2 lần. Cần ghép cặp bằng:

- thời gian xuất hiện gần nhau,
- vị trí gần nhau sau khi quy chiếu,
- class giống nhau,
- hướng di chuyển tương tự,
- đặc trưng hình ảnh nếu cần.

Hàm ghép cặp có thể là:

$$score(a, b) = \alpha \cdot sim_{time} + \beta \cdot sim_{position} + \gamma \cdot sim_{class} + \delta \cdot sim_{trajectory}$$

Nếu $score > threshold$ thì xem là cùng một xe.

E3. Loại trùng

Sau khi matching:

- chỉ giữ 1 bản ghi đại diện cho xe,
- hoặc gộp thành 1 thực thể thống nhất.

Đây là bước quyết định để giảm sai số mật độ và lưu lượng.

5) Hợp nhất dữ liệu có trọng số tại node

Đây là khối “Hợp nhất dữ liệu loại trùng, hợp nhất có trọng số”.

Sau khi mỗi camera cho ra:

- D_i : mật độ,
- V_i : vận tốc,
- q_i : chất lượng ảnh,
- r_i : độ tin cậy camera,
- o_i : mức che khuất hoặc mức chồng lấn.

Ta xây dựng trọng số:

$$w_i = \alpha q_i + \beta r_i + \gamma o_i$$

hoặc chuẩn hóa:

$$w_i = \frac{\alpha q_i + \beta r_i + \gamma o_i}{\sum_j (\alpha q_j + \beta r_j + \gamma o_j)}$$

F1. Hợp nhất mật độ

$$D_{node} = \frac{\sum_i w_i D_i}{\sum_i w_i}$$

F2. Hợp nhất vận tốc

$$V_{node} = \frac{\sum_i w_i V_i}{\sum_i w_i}$$

F3. Tính mức ùn tắc node

Có thể định nghĩa:

$$CongestionScore = \lambda_1 D_{node} + \lambda_2 \left(\frac{1}{V_{node} + \epsilon} \right)$$

Hoặc gán nhãn:

- Thoáng
- Trung bình
- Đông
- ùn tắc

6) Agent cục bộ tại node nên làm gì?

Đây là phần để kiến trúc **Node-Agent-Edge** có AI, không chỉ là pipeline camera đơn thuần.

Agent tại mỗi node có các nhiệm vụ:

G1. Thu nhận dữ liệu đa camera

- đọc stream,
- kiểm tra trạng thái camera,
- phát hiện camera lỗi.

G2. Điều phối xử lý AI

- chọn FPS xử lý phù hợp,
- giảm tải khi tài nguyên yếu,
- ưu tiên camera quan trọng.

G3. Hợp nhất dữ liệu node

- dedup,
- weighted fusion,
- tạo trạng thái giao thông cục bộ.

G4. Ra quyết định cục bộ

Ví dụ:

- cảnh báo ùn tắc,
- đề xuất tăng thời gian đèn xanh,
- gửi cảnh báo camera lỗi,
- kích hoạt chế độ dự phòng.

G5. Giao tiếp với trung tâm

Gửi:

- node_id
- timestamp
- density
- velocity
- congestion level
- camera health
- processing latency
- confidence.

7) Tầng trung tâm hiển thị bản đồ giao thông website

Đây là khối “**Hiển thị Bản đồ giao thông**”.

H1. Dashboard trung tâm

Mỗi node trên bản đồ có:

- màu sắc theo mức ùn tắc,
- vận tốc trung bình,

- mật độ,
- số camera đang hoạt động,
- độ tin cậy node.

H2. Các lớp hiển thị

- lớp camera,
- lớp node,
- lớp đoạn đường,
- heatmap mật độ,
- vector hướng lưu thông.

H3. Công nghệ gợi ý

- Backend: FastAPI
- Database: PostgreSQL / TimescaleDB / MongoDB
- Realtime: MQTT / Kafka / WebSocket
- Frontend map: Leaflet / Mapbox / OpenLayers
- Dashboard: hoặc React.

8) Kiến trúc kỹ thuật hiện thực đề xuất **Lý thuyết** → Tại Node

Mỗi node gồm các service:

Service 1. Camera Ingestion Service

- lấy RTSP stream,
- trích frame,
- đồng bộ timestamp.

Service 2. Preprocessing Service

- ROI,
- resize,
- blur checking,
- brightness checking.

Service 3. Detection & Tracking Service

- YOLO
- ByteTrack/DeepSORT

Service 4. Traffic Estimation Service

- đếm xe,
- phân loại xe,
- tính D_i ,

- tính V_i .

Service 5. Overlap Resolution Service

- phát hiện đối tượng trùng,
- loại trùng giữa camera.

Service 6. Fusion Agent

- tính trọng số,
- hợp nhất mật độ và vận tốc,
- suy luận trạng thái node.

Service 7. Edge Communication Service

- gửi JSON/MQTT/Kafka message lên server.

9) Cấu trúc dữ liệu đầu ra nên chuẩn hóa

Ví dụ mỗi camera (TomTom) trả về:

```
{
  "timestamp": "2026-04-13T10:30:00",
  "node_id": "N01",
  "camera_id": "C1",
  "vehicle_counts": {
    "motorcycle": 25,
    "car": 12,
    "bus": 1,
    "truck": 2
  },
  "density": 0.36,
  "velocity": 28.4,
  "image_quality": 0.82,
  "reliability": 0.90,
  "occlusion_score": 0.75
}
```

Sau hợp nhất ở node:

```
{
  "timestamp": "2026-04-13T10:30:00",
  "node_id": "N01",
  "fused_density": 0.31,
  "fused_velocity": 26.8,
  "congestion_level": "medium",
  "confidence": 0.87,
  "active_cameras": 3,
  "latency_ms": 420
}
```

10) Các chỉ số đánh giá hiệu năng nên có

Vì đề tài có chữ “**Performance Evaluation of the Node-Agent-Edge Architecture**”, thầy phải đánh giá ít nhất 4 nhóm chỉ số:

Nhóm 1. Độ chính xác giao thông

- sai số đếm xe,
- sai số mật độ,
- sai số vận tốc,
- sai số phát hiện ùn tắc.

Nhóm 2. Hiệu năng xử lý

- latency/frame,
- latency/node,
- FPS xử lý,
- thời gian phản hồi cảnh báo.

Nhóm 3. Hiệu quả truyền thông

- băng thông sử dụng,
- dung lượng dữ liệu gửi về trung tâm,
- tần suất gửi.

Nhóm 4. Độ bền hệ thống

- khi mất 1 camera thì hệ thống còn chạy tốt không,
- khi 1 node bị lỗi thì hệ thống có suy giảm bao nhiêu,
- khi ảnh mờ/nhiều thì độ chính xác giảm bao nhiêu.

11) Các kịch bản thí nghiệm nên làm

Kịch bản 1. So sánh 1 camera vs đa camera hợp nhất

Mục tiêu:

- chứng minh fusion tốt hơn single camera.

Kịch bản 2. Có dedup vs không dedup

Mục tiêu:

- chứng minh xử lý vùng chồng lấn giúp giảm đếm trùng.

Kịch bản 3. Hợp nhất có trọng số vs trung bình đơn giản

Mục tiêu:

- chứng minh weighted fusion ổn định hơn.

Kịch bản 4. Edge processing vs gửi toàn bộ video lên cloud

Mục tiêu:

- chứng minh Node-Agent-Edge giảm độ trễ và giảm băng thông.

Kịch bản 5. Hồng 1 camera

Mục tiêu:

- chứng minh hệ thống vẫn hoạt động được và còn khả năng chống chịu.

12) Pipeline thuật toán viết gọn cho DACN/DATN

Bước 1. Thu nhận video đồng thời từ nhiều camera tại mỗi node biên TomTom

Bước 2. Tiền xử lý dữ liệu ảnh, chuẩn hóa khung hình và trích vùng quan tâm

Bước 3. Phát hiện và phân loại phương tiện bằng mô hình AI

Bước 4. Theo dõi đối tượng qua chuỗi khung hình để ước lượng quỹ đạo và vận tốc

Bước 5. Tính toán các chỉ số giao thông cục bộ tại từng camera, gồm mật độ và vận tốc

Bước 6. Xác định vùng chồng lấn giữa các camera và loại bỏ các đối tượng bị đếm trùng

Bước 7. Hợp nhất dữ liệu đa camera bằng cơ chế gán trọng số theo chất lượng ảnh, độ tin cậy cảm biến và mức che khuất

Bước 8. Tạo trạng thái giao thông hợp nhất tại node biên thông qua AI Agent cục bộ

Bước 9. Truyền kết quả đã hợp nhất từ các node biên lên máy chủ trung tâm

Bước 10. Hiển thị bản đồ giao thông theo thời gian thực Web và đánh giá hiệu năng kiến trúc Node-Agent-Edge

13) Gợi ý công nghệ để làm thật DACN DATN

AI/Computer Vision

- Python
- OpenCV
- Ultralytics YOLO
- ByteTrack / DeepSORT

Edge

- Jetson / Mini PC / PC biên
- Docker
- FastAPI
- MQTT

Backend trung tâm

- FastAPI / Django
- PostgreSQL / MongoDB
- Redis
- Kafka hoặc MQTT broker

Visualization

- Streamlit để demo nhanh
- React + Leaflet/Mapbox để làm hệ thống đẹp hơn

14) Các em chú ý **bổ sung hen**

Thiếu 1: Tracking

Chỉ detection thôi là chưa đủ. Muốn tính vận tốc và tránh đếm lặp thì **tracking là bắt buộc**.

Thiếu 2: Đồng bộ thời gian và ánh xạ không gian

Muốn hợp nhất đa camera thì phải có:

- time synchronization,
- calibration hoặc homography.

Thiếu 3: Agent logic

Tên đề tài có **Node-Agent-Edge**, nên phải làm rõ:

- Agent nhận gì,
- Agent quyết định gì,
- Agent gửi gì,
- Agent phản ứng thế nào khi camera lỗi hoặc dữ liệu kém tin cậy.

Kiến trúc module chi tiết kiểu triển khai bằng Python + FastAPI + YOLO + ReactJS